SR UNIVERSITY

AI Assistant Coding
Assignment-6.2

Name of Student : G.Varshith Reddy
Enrollment_No : 2303A52287
Batch_No : 37

Task Description-1 (Classes – Data Validation)

• Prompt AI to generate a Student class with attributes: name, roll_no, and marks. Add a method is_pass() that returns whether the student has passed (marks ≥ 40).

Expected Output-1

• Python class with constructor and validation logic
• Clear method implementation returning pass/fail status
• Students analyze correctness and clarity of class design

Prompt:

Generate student class with name, roll number and marks. Add a method is pass that returns whether the student has passed. Marks should be greater than or equal to 40

Code:

```
class Student:
    def __init__(self, name, roll_number, marks):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks

    def is_pass(self):
        return self.marks >= 40

    def __str__(self):
        return f"Student(Name: {self.name}, Roll: {self.roll_number}, Marks: {self.marks})"
```

Output:

```
john doe is: Pass
jane smith is: Fail
```

Justification:

Creates a new student instance with the provided academic information.
Evaluates student performance against the passing threshold of 40 marks.
Provides a formatted display of the student

Task Description-2 (Loops – Pattern Generation)

• Ask AI to generate a function that prints a right-angled triangle star pattern using a for loop. Then regenerate the same pattern using a while loop.

Expected Output-2

• Correct pattern output using both loop types
• Logical loop structure with proper conditions

Prompt:

Generate a function that prints right-angle triangle star pattern using for loop and while loop

Code:

```python
def print_triangle_for_loop(rows):
    """
    Print a right-angle triangle star pattern using for loop

    Args:
        rows (int): Number of rows in the triangle
    """
    for i in range(1, rows + 1):
        for j in range(i):
            print("*", end="")
        print()  # New line after each row


def print_triangle_while_loop(rows):
    """
    Print a right-angle triangle star pattern using while loop

    Args:
        rows (int): Number of rows in the triangle
    """
    i = 1
    while i <= rows:
        j = 0
        while j < i:
            print("*", end="")
            j += 1
        print()  # New line after each row
        i += 1


# Example usage
if __name__ == "__main__":
    rows = 5

    print("Triangle using for loop:")
    print_triangle_for_loop(rows)

    print("\nTriangle using while loop:")
    print_triangle_while_loop(rows)
```

Output:

```
Triangle using for loop:
*
**
***
****
*****

Triangle using while loop:
*
**
***
****
*****
```

Justification:

Print a right-angle triangle star pattern using while loop. This function creates a triangle pattern where each row contains an increasing number of asterisks (*), starting with 1 asterisk in the first row and ending with 'rows' number of asterisks in the last row.

rows (int): Number of rows in the triangle. Must be a positive integer.

Returns:

None: This function prints directly to stdout and doesn't return a value.

Example:

```
>>> print_triangle_while_loop(4)
*
**
***
****
```

Note:

Uses nested while loops for demonstration purposes. The outer loop controls the number of rows, while the inner loop prints the asterisks for each row.

Task Description-3 (Conditional Statements – Number Analysis)

- Ask AI to write a function that checks whether a given number is positive, negative, or zero using if-elif-else. Test the function with multiple inputs.

Expected Output-3

- Function correctly classifies numbers
- Proper handling of all conditions
- Students analyze decision logic

Prompt:

Code:

```python
def extract_digits(text: str) -> str:
    """Extract and return all digit characters from an alphanumeric string."""
    return "".join(ch for ch in text if ch.isdigit())


if __name__ == "__main__":
    print('extract_digits("a1b2c3") ->', extract_digits("a1b2c3"))
    print('extract_digits("abc2025xyz") ->', extract_digits("abc2025xyz"))
    print('extract_digits("x9y") ->', extract_digits("x9y"))
```

Output:

```
extract_digits("a1b2c3") -> 123
extract_digits("abc2025xyz") -> 2025
extract_digits("x9y") -> 9
```

Justification:

Few-shot prompting uses multiple examples to show clear input–output patterns. These examples help the AI identify the exact rule for extracting digits. As a result, the generated function is more precise and reliable.

Task Description-4

- **Compare zero-shot vs few-shot prompting for generating a function that counts the number of vowels in a string.**

Expected Output-4

- **Output comparison + student explanation on how examples helped the model.**

Prompt:

Zero-Shot Prompt

Write a Python function to count the number of vowels in a string.

Few-Shot Prompt

Example 1: "hello" → 2
Example 2: "AI Tools" → 4
Example 3: "xyz" → 0

Write a Python function to count vowels in a string.

Code:

Zero-Shot Prompt

```python
def count_vowels(text: str) -> int:
    """Return the number of vowels (a, e, i, o, u) in a string."""
    vowels = {"a", "e", "i", "o", "u"}
    return sum(1 for ch in text.lower() if ch in vowels)


if __name__ == "__main__":
    print("Vowels -> ", count_vowels("hello"))
    print("Vowels -> ", count_vowels("AI Code"))
```

Few-Shot Prompt

```python
def count_vowels(s):
    vowels = set("aeiouAEIOU")
    return sum(1 for ch in s if ch in vowels)

if __name__ == "__main__":
    print('count_vowels("hello") ->', count_vowels("hello"))
    print('count_vowels("AI Tools") ->', count_vowels("AI Tools"))
    print('count_vowels("xyz") ->', count_vowels("xyz"))
```

Output:

Zero-Shot Prompt

```
Vowels ->  2
Vowels ->  4
```

Few-Shot Prompt

```
count_vowels("hello") -> 2
count_vowels("AI Tools") -> 4
count_vowels("xyz") -> 0
```

Comparison & Explanation:

Aspect	Zero-Shot	Few-Shot
Logic clarity	Basic loop	Optimized logic

Accuracy	Correct	Correct
Code quality	Simple	Cleaner & concise
Guidance	No examples	Clear behavior from examples

Justification:

Few-shot prompting improves output quality because examples clarify expected behavior, edge cases, and efficiency. The model generates more refined and optimized solutions when patterns are visible.

Task Description-5

- **Use few-shot prompting with 3 sample inputs to generate a function that determines the minimum of three numbers without using the built-in min() function.**

Expected Output-5

- **A function that handles all cases with correct logic based on example patterns.**

Prompt:

Example 1: (3, 7, 5) → 3
Example 2: (10, 2, 8) → 2
Example 3: (4, 4, 9) → 4
Write a Python function to find the minimum of three numbers without using min()

Code:

```python
def min_of_three(a: int | float, b: int | float, c: int | float) -> int | float:
    """Return the minimum of three numbers without using min()."""
    smallest = a
    if b < smallest:
        smallest = b
    if c < smallest:
        smallest = c
    return smallest

if __name__ == "__main__":
    print("min_of_three(3, 7, 5) ->", min_of_three(3, 7, 5))
    print("min_of_three(10, 2, 8) ->", min_of_three(10, 2, 8))
    print("min_of_three(4, 4, 9) ->", min_of_three(4, 4, 9))
```

OUTPUT:

```
min_of_three(3, 7, 5) -> 3
min_of_three(10, 2, 8) -> 2
min_of_three(4, 4, 9) -> 4
```

JUSTIFICATION:

Few-shot examples help the AI understand comparison logic and edge cases without using built-in functions. The model learns the decision pattern from examples and applies it correctly. This results in accurate handling of all possible inputs.